

CMPT 354
Database Systems I

*SQL Modifications and
Transactions*

Martin Ester
Simon Fraser University
Fall 2023

Introduction



- SQL provides three operations that modify the instance (state) of a DB:
 - INSERT: inserts new tuple(s),
 - DELETE: deletes existing tuples(s), and
 - UPDATE: updates attribute value(s)s of existing tuple(s).
- Individual modifications may yield an inconsistent DB state, and only a sequence of modifications (*transaction*) may lead again to a consistent state.
- The DBS ensures certain properties of a transaction that guarantee the consistency of the resulting DB state.

Insertion



```
INSERT INTO R (A1, . . . , An)  
VALUES (v1, . . . , vn);
```

- Inserts a single tuple with values v_i for attributes A_i into table R.

```
INSERT INTO Sailors (sid, sname, rating, age)  
VALUES (69, 'mike', 2, 20);
```

- If values are not provided for all attributes, NULL values will be inserted.
- Short hand if all attribute values are given:

```
INSERT INTO Sailors  
VALUES (69, 'mike', 2, 20);
```

- Values need to be provided in the order of the corresponding attributes.

Insertion



```
INSERT INTO R (A1, . . . , An)
<subquery> ;
```

- Inserts a set of tuples (relation) with values for attributes A_i into table R , as specified by a subquery.

```
INSERT INTO Sailors (sid)
SELECT DISTINCT R.sid
FROM Reserves R
WHERE R.sid NOT IN
      (SELECT sid
       FROM Sailors);
```

- The subquery is completely evaluated before the first tuple is inserted.

Deletion



DELETE FROM R
WHERE <condition> ;

- Deletes the set (!) of all tuples from R which satisfy the condition of the WHERE clause.

DELETE FROM Sailors // one tuple
WHERE sid = 69;

DELETE FROM Sailors // multiple tuples
WHERE rating < 3;

- Cannot directly specify a tuple to be deleted as is possible for insertions.

Update



```
UPDATE R
SET <new value assignments>
WHERE <condition>;
```

- Updates attributes in the specified manner for all R tuples satisfying the given condition.

```
UPDATE Sailors
SET age = age + 1;
```

```
UPDATE Sailors
SET rating = rating * 1.1, age = age + 1
WHERE age < 30 and sid IN
    (SELECT R.sid
     FROM Reserves R);
```

Transactions

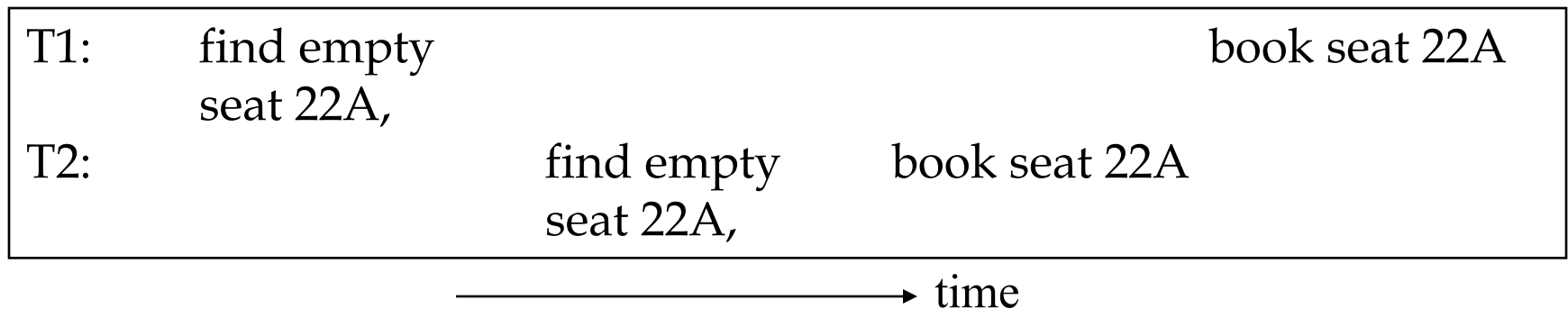


- So far, we have implicitly assumed that there is only one DB user who executes one SQL statement at a time.
- In reality, a DBS may have many concurrent users.
- Each user may issue a sequence of SQL statements that form a logical unit (*transaction*).
- The DBS is in charge of ordering the SQL statements from different users in a way (*serializable*) that produces the same results as if the statements would have been executed in a single user scenario.

Serializability



- Consider two users who simultaneously want to book a seat on a certain flight: they first find an empty seat and then book it (set it occupied).
- In an unconstrained system, their operations might be executed in the following order:



- In the end, who gets seat 22A?

Serializability



- To avoid such a problem, we need to consider the sequence of statements of a user transaction as a *unit*.
- Statements from two different user transactions must be ordered in a way that is equivalent to both transactions being performed serially in either order (transaction 1 before transaction 2 or transaction 2 before transaction 1).
- In our example, either user 1 or user 2 would get seat 22A. The other user would see 22A as occupied and would have to find another seat.

Atomicity



- So far, we have also assumed that all SQL statements are executed correctly.
- In reality, various types of system errors can occur during the execution of a user transaction.
- At the time of a system crash, transactions can be incomplete: some, but not all of their SQL statements have been executed.

Atomicity



- Consider a bank transaction T:

T: $A=A+100, \quad B=B-100$

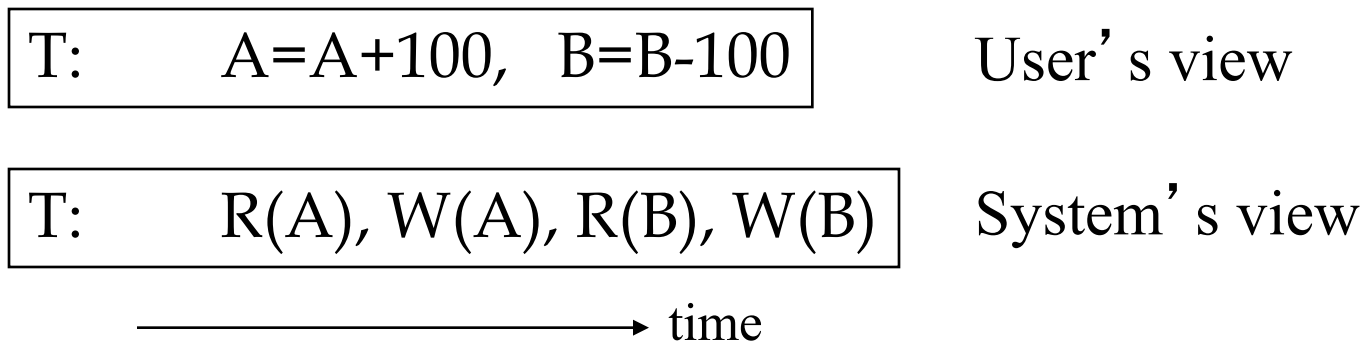
—————→ time

- T is transferring \$100 from B' s account to A' s account.
- What if the system crashes right after the first statement of T has been executed, i.e. the second statement is not executed?
- The DBS has to ensure that every transaction is treated as an *atomic* unit, i.e. either all or none of its SQL statements are executed.

Transactions



- A user's program may carry out many operations on the data retrieved from the database, but the DBMS is only concerned about what data is read/written from/to the database.
- A *transaction* is the DBMS's abstract view of a user program: a sequence of DB reads (R) and writes (W).



Serializability



- *Serial schedule*: Schedule that does not interleave the SQL statements of different transactions.
- *Equivalent schedules*: For any database state, the effect of executing the first schedule is identical to the effect of executing the second schedule:
 - The resulting DB instance / state.
 - The result of read operations, i.e. what the user sees.
- *Serializable schedule*: A schedule that is equivalent to some serial schedule.

Serializability



- Serializability is normally ensured by *locking* the DB objects read or written.
- Before reading or writing a DB object (table or tuple), the transaction needs to obtain the appropriate lock:
 - *Shared locks* for reading,
 - *Exclusive locks* for writing,
 - *View locks* to prevent new tuples from being returned by a repeated reading.
- The same transaction with lock requests:

T: S(A), R(A), X(A), W(A), S(B), R(B), X(B), W(B)
--

S(. . .): request a shared lock,

X(. . .): request an exclusive lock.

Serializability



- Locks are normally released only at the end of a transaction.
- If a transaction cannot get the lock it has requested, it needs to wait until it is released by the other transaction currently holding it.
- Without locks:

T1:	R(A)		W(A)
T2:		R(A)	W(A)

- With locks:

T1:	S(A), R(A), X(A),	W(A), Release locks,
T2:	S(A),	R(A), X(A), W(A)

Transactions in SQL



- By default, each SQL statement (any query or modification of the database or its schema) is treated as a separate transaction.
 - Transaction includes the effects of triggers.
- Transactions can also be defined explicitly.
`START TRANSACTION;`
 <sequence of SQL statements>
`COMMIT;`
or `ROLLBACK;`
- COMMIT makes all modifications of the transaction permanent, ROLLBACK undoes all DB modifications made by the transaction.

Read-Only Transactions



- A transaction that reads only (and does not write the DB) is easy to serialize with other read-only transactions.
- Only shared locks need to be set.
- This means that read-only transactions do not need to wait for each other, and the throughput of the DBS can be maximized.
- To specify the next transaction as read-only:
`SET TRANSACTION READ ONLY;`
- Can the query compiler not automatically detect whether a transaction is read-only?

Dirty Reads



- *Dirty data* is data that has been modified by a transaction that has not yet committed.
- If that transaction is rolled back after another transaction has read its dirty data, a non-serializable schedule results.
- Consider the following schedule of T1 which wants to modify database objects A and B and T2 which wants to modify database object A.

T1:	R(A), W(A),	R(B), W(B), Rollback
T2:	R(A), W(A), Commit	

- T2 reads dirty data, written by T1, before T1 commits.

Dirty Reads



- The effect of this schedule is that T2 modifies A based on the version of A produced by T1.
- The effect of the serial schedule T1, T2 is that T2 modifies A based on the original version of A.
- The effect of the serial schedule T2, T1 is that T2 modifies A based on the original version of A.
- Therefore, the interleaved schedule is not serializable.
- With locks, we get the following serializable schedule:

T1:	S(A), R(A), X(A),	W(A), Release locks,
T2:	S(A),	R(A), X(A), W(A)

Isolation Levels



- The SQL default isolation level ensures serializability.
- There are scenarios where a weaker isolation level may be acceptable (and more efficient!).
- SQL allows you to specify four different isolation levels for a transaction.
`SET TRANSACTION ISOLATION LEVEL . . . ;`
- The *isolation level* of a transaction defines what data *that* transaction may see.
- Note that other, concurrent transactions may be executed at *different* isolation levels.

Isolation Levels



- The semantics of the four different isolation levels is defined as follows:

Isolation Level	Write locks	Read locks	View locks
Read Uncommitted	No	No	No
Read Committed	Yes	No	No
Repeatable Reads	Yes	Yes	No
Serializable	Yes	Yes	Yes

Transactions



- Requirements for transactions:
 - **Atomicity**: “all or nothing”,
 - **Consistency**: transforms consistent DB state into another consistent DB state,
 - **Independence**: from all other transactions (serializability),
 - **Durability**: survives any system crashes.
- These requirements are called **ACID** properties of transactions.

Summary



- A transaction consists of a sequence of read / write operations.
- The DBMS guarantees the atomicity, consistency, independence and durability of transactions.
- Serializability guarantees independence of transactions.
- Lower isolation levels can be specified. They may be acceptable and more efficient in certain scenarios.